

UNIT-1

Preliminary Concepts

Background

- Frankly, we didn't have the vaguest idea how the thing [FORTRAN language and compiler] would work out in detail. ...We struck out simply to optimize the object program, the running time, because most people at that time believed you couldn't do that kind of thing. They believed that machined-coded programs would be so inefficient that it would be impractical for many applications.
- John Backus, unexpected successes are common – the browser is another example of an unexpected success

1.1 Reasons for Studying Concepts of Programming Languages- CO1

- Increased ability to express ideas
- Improved background for choosing appropriate languages
- Increased ability to learn new languages
- Better understanding of significance of implementation
- Overall advancement of computing

1.2 Programming Domains – CO1

- Scientific applications
 - Large number of floating point computations
 - Fortran
- Business applications
 - Produce reports, use decimal numbers and characters
 - COBOL
- Artificial intelligence
 - Symbols rather than numbers manipulated
 - LISP
- Systems programming
 - Need efficiency because of continuous use
 - C
- Web Software
 - Eclectic collection of languages: markup (**e.g.**, XHTML), scripting (**e.g.**, PHP), general-purpose (**e.g.**, Java)

1.2 Language Evaluation Criteria – CO1, CO2

- Readability : the ease with which programs can be read and understood
- Writability : the ease with which a language can be used to create programs
- Reliability : conformance to specifications (i.e., performs to its specifications)
- Cost : the ultimate total cost

Readability

- Overall simplicity
 - A manageable set of features and constructs
 - Few feature multiplicity (means of doing the same operation)
 - Minimal operator overloading
- Orthogonality
 - A relatively small set of primitive constructs can be combined in a relatively small number of ways
 - Every possible combination is legal
- Control statements
 - The presence of well-known control structures (**e.g.**, while statement)
- Data types and structures
 - The presence of adequate facilities for defining data structures
- Syntax considerations
 - Identifier forms: flexible composition
 - Special words and methods of forming compound statements
 - Form and meaning: self-descriptive constructs, meaningful keywords

Writability

- Simplicity and Orthogonality
 - Few constructs, a small number of primitives, a small set of rules for combining them
- Support for abstraction
 - The ability to define and use complex structures or operations in ways that allow details to be ignored
- Expressivity
 - A set of relatively convenient ways of specifying operations
 - Example: the inclusion of for statement in many modern languages

Reliability

- Type checking
 - Testing for type errors
- Exception handling
 - Intercept run-time errors and take corrective measures
- Aliasing
 - Presence of two or more distinct referencing methods for the same memory location
- Readability and writability
 - A language that does not support “natural” ways of expressing an algorithm will necessarily use “unnatural” approaches, and hence reduced reliability

Cost

- Training programmers to use language
- Writing programs (closeness to particular applications)
- Compiling programs
- Executing programs

- Language implementation system: availability of free compilers
- Reliability: poor reliability leads to high costs
- Maintaining programs

Others

- Portability
 - The ease with which programs can be moved from one implementation to another
- Generality
 - The applicability to a wide range of applications
- Well-definedness
 - The completeness and precision of the language's official definition

1.3 Influences on Language Design - CO3

- Computer Architecture
 - Languages are developed around the prevalent computer architecture, known as the *von Neumann* architecture
- Programming Methodologies
 - New software development methodologies (**e.g.**, object-oriented software development) led to new programming paradigms and by extension, new programming languages

Computer Architecture

- Well-known computer architecture: Von Neumann
- Imperative languages, most dominant, because of von Neumann computers
 - Data and programs stored in memory
 - Memory is separate from CPU
 - Instructions and data are piped from memory to CPU
 - Basis for imperative languages
- Variables model memory cells
- Assignment statements model piping
- Iteration is efficient

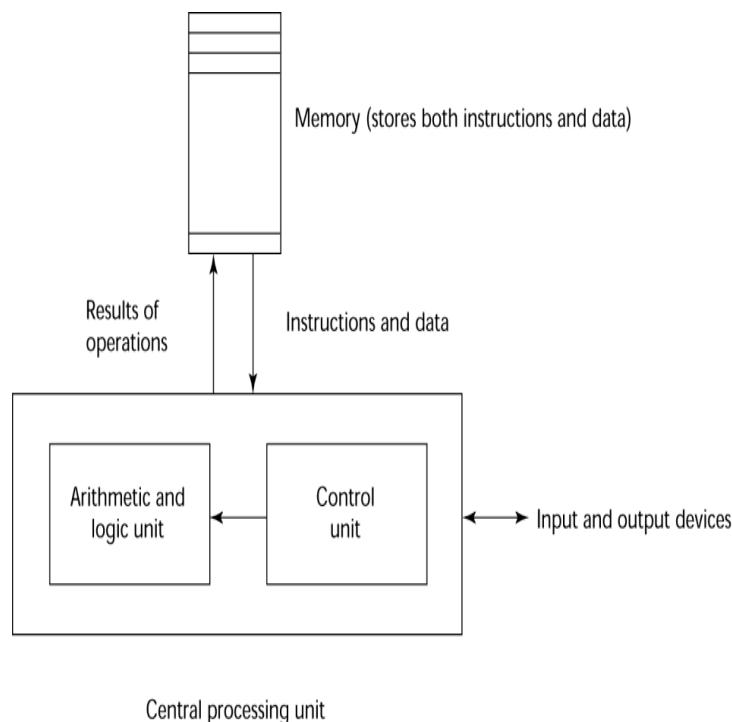


Figure 1.1 The von Neumann Computer Architecture

Programming Methodologies

- 1950s and early 1960s: Simple applications; worry about machine efficiency
- Late 1960s: People efficiency became important; readability, better control structures
 - structured programming
 - top-down design and step-wise refinement
- Late 1970s: Process-oriented to data-oriented
 - data abstraction
- Middle 1980s: Object-oriented programming
 - Data abstraction + inheritance + polymorphism

1.4 Language Categories – CO1

- Imperative
 - Central features are variables, assignment statements, and iteration
 - Examples: C, Pascal
- Functional
 - Main means of making computations is by applying functions to given parameters
 - Examples: LISP, Scheme
- Logic
 - Rule-based (rules are specified in no particular order)
 - Example: Prolog
- Object-oriented
 - Data abstraction, inheritance, late binding
 - Examples: Java, C++
- Markup
 - New; not a programming per se, but used to specify the layout of information in Web documents
 - Examples: XHTML, XML

Language Design Trade-Offs

- Reliability vs. cost of execution
 - Conflicting criteria
 - Example: Java demands all references to array elements be checked for proper indexing but that leads to increased execution costs
- Readability vs. writability
 - Another conflicting criteria
 - Example: APL provides many powerful operators (and a large number of new symbols), allowing complex computations to be written in a compact program but at the cost of poor readability
- Writability (flexibility) vs. reliability
 - Another conflicting criteria
 - Example: C++ pointers are powerful and very flexible but not reliably used

1.5 Implementation Methods -CO2

- Compilation
 - Programs are translated into machine language
- Pure Interpretation
 - Programs are interpreted by another program known as an interpreter
- Hybrid Implementation Systems
 - A compromise between compilers and pure interpreters

Compilation

- Translate high-level program (source language) into machine code (machine language)
- Slow translation, fast execution
- Compilation process has several phases:
 - lexical analysis: converts characters in the source program into lexical units
 - syntax analysis: transforms lexical units into *parse trees* which represent the syntactic structure of program
 - Semantics analysis: generate intermediate code
 - code generation: machine code is generated

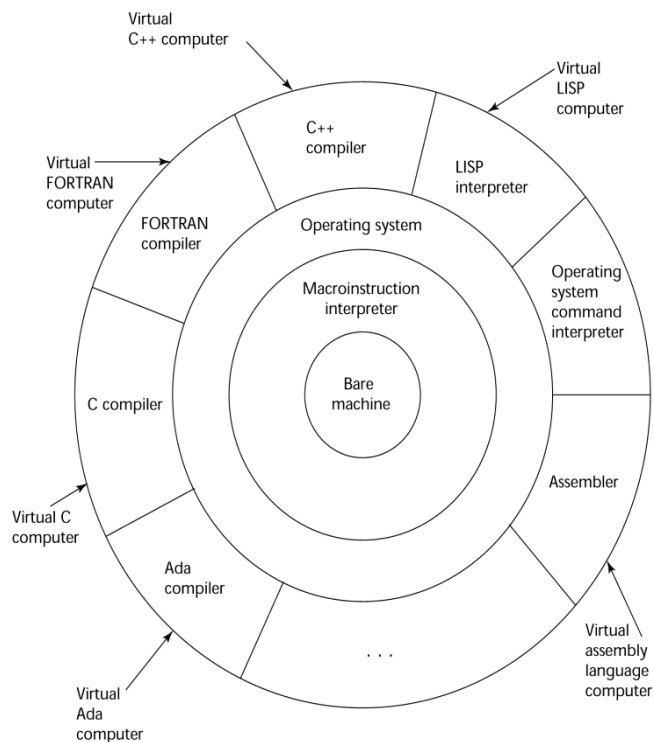


Figure 1.2 Layered View of Computer: The operating system and language implementation are layered over Machine interface of a computer

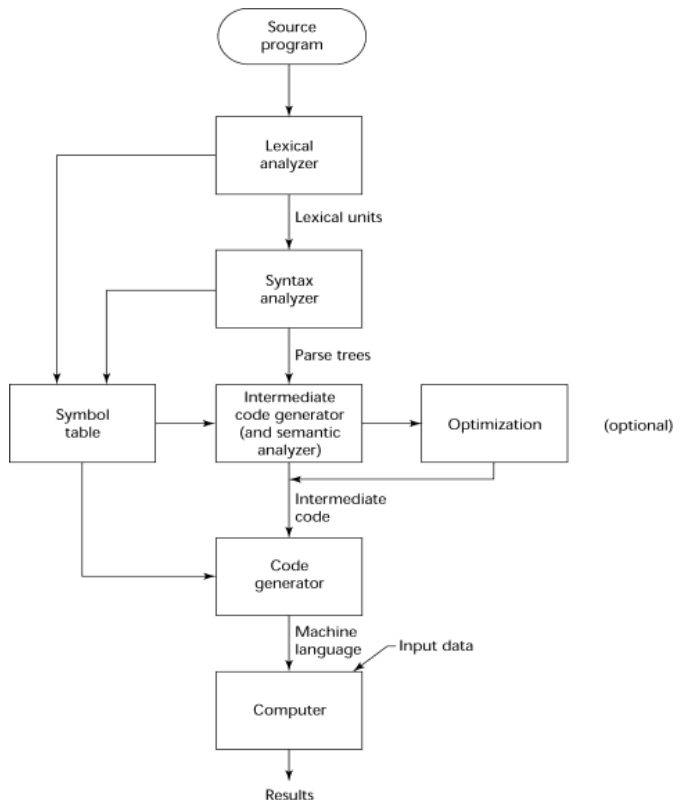


Figure 1.3 The Compilation Process

Additional Compilation Terminologies

- Load module (executable image): the user and system code together
- Linking and loading: the process of collecting system program and linking them to user program

Execution of Machine Code

- Fetch-execute-cycle (on a von Neumann architecture)

initialize the program counter

repeat forever

fetch the instruction pointed by the counter

increment the counter

decode the instruction

execute the instruction

end repeat

Von Neumann Bottleneck

- Connection speed between a computer's memory and its processor determines the speed of a computer
- Program instructions often can be executed a lot faster than the above connection speed; the connection speed thus results in a *bottleneck*
- Known as von Neumann bottleneck; it is the primary limiting factor in the speed of computers

Pure Interpretation

- No translation
- Easier implementation of programs (run-time errors can easily and immediately displayed)
- Slower execution (10 to 100 times slower than compiled programs)
- Often requires more space
- Becoming rare on high-level languages

- Significantly comeback with some latest web scripting languages (e.g., JavaScript)

Hybrid Implementation Systems

- A compromise between compilers and pure interpreters
- A high-level language program is translated to an intermediate language that allows easy interpretation
- Faster than pure interpretation
- Examples
 - Perl programs are partially compiled to detect errors before interpretation
 - Initial implementations of Java were hybrid; the intermediate form, *byte code*, provides portability to any machine that has a byte code interpreter and a runtime system (together, these are called *Java Virtual Machine*)

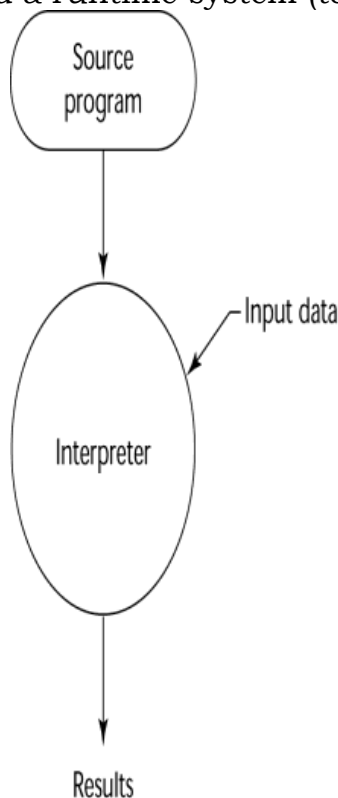


Figure 1.4 Pure Interpretation

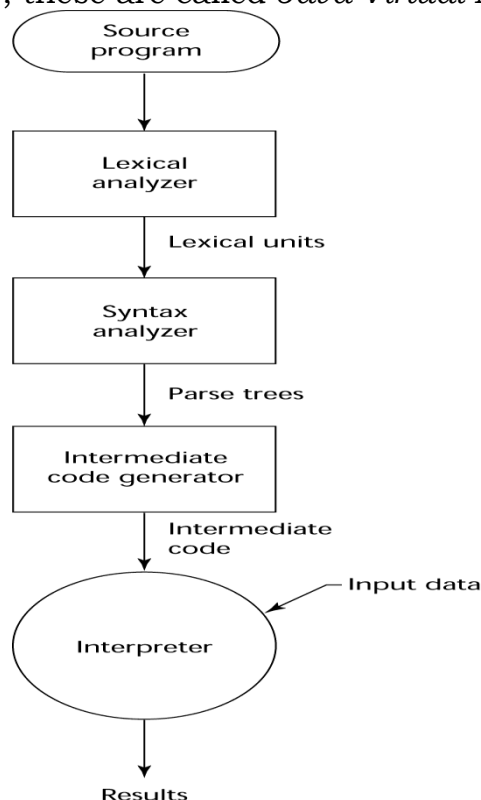


Figure 1.5 Hybrid Implementation

Just-in-Time Implementation Systems

- Initially translate programs to an intermediate language
- Then compile intermediate language into machine code
- Machine code version is kept for subsequent calls
- JIT systems are widely used for Java programs
- .NET languages are implemented with a JIT system

1.6 Preprocessors - CO2

- Preprocessor macros (instructions) are commonly used to specify that code from another file is to be included
- A preprocessor processes a program immediately before the program is compiled to expand embedded preprocessor macros
- A well-known example: C preprocessor
 - expands #include, #define, and similar macros

Syntax and Semantics

Introduction

- **Syntax:** the form or structure of the expressions, statements, and program units
- **Semantics:** the meaning of the expressions, statements, and program units
- Syntax and semantics provide a language's definition
 - Users of a language definition
 - Other language designers
 - Implementers
 - Programmers (the users of the language)

1.7 The General Problem of Describing Syntax – CO1

- A *sentence* is a string of characters over some alphabet
- A *language* is a set of sentences
- A *lexeme* is the lowest level syntactic unit of a language (**e.g.**, *, sum, begin)
- A *token* is a category of lexemes (**e.g.**, identifier)
- **Languages Recognizers**
 - A recognition device reads input strings of the language and decides whether the input strings belong to the language
 - Example: syntax analysis part of a compiler
- **Languages Generators**
 - A device that generates sentences of a language
 - One can determine if the syntax of a particular sentence is correct by comparing it to the structure of the generator

1.8 Formal Methods of Describing Syntax – CO1,CO2

- Backus-Naur Form and Context-Free Grammars
 - Most widely known method for describing programming language syntax
- Extended BNF
 - Improves readability and writability of BNF
- Grammars and Recognizers

Backus-Naur Form and Context-Free Grammars

- Context-Free Grammars
- Developed by Noam Chomsky in the mid-1950s
- Language generators, meant to describe the syntax of natural languages
- Define a class of languages called context-free languages

Backus-Naur Form (BNF)

- Backus-Naur Form (1959)
 - Invented by John Backus to describe ALGOL 58
 - BNF is equivalent to context-free grammars
 - BNF is a *metalanguage* used to describe another language
 - In BNF, abstractions are used to represent classes of syntactic structures-- they act like syntactic variables (also called *nonterminal symbols*)

BNF Fundamentals

- Non-terminals: BNF abstractions
- Terminals: lexemes and tokens
- Grammar: a collection of rules
 - Examples of BNF rules:

$\langle \text{ident_list} \rangle \rightarrow \text{identifier} \mid \text{identifier}, \langle \text{ident_list} \rangle$
 $\langle \text{if_stmt} \rangle \rightarrow \text{if } \langle \text{logic_expr} \rangle \text{ then } \langle \text{stmt} \rangle$

BNF Rules

- A rule has a left-hand side (LHS) and a right-hand side (RHS), and consists of *terminal* and *nonterminal* symbols
- A grammar is a finite nonempty set of rules
- An abstraction (or nonterminal symbol) can have more than one RHS

$$\langle \text{stmt} \rangle \rightarrow \langle \text{single_stmt} \rangle \mid \text{begin } \langle \text{stmt_list} \rangle \text{ end}$$

Describing Lists

- Syntactic lists are described using recursion

$$\langle \text{ident_list} \rangle \rightarrow \text{ident} \mid \text{ident}, \langle \text{ident_list} \rangle$$
- A derivation is a repeated application of rules, starting with the start symbol and ending with a sentence (all terminal symbols)

An Example Grammar

$\langle \text{program} \rangle \rightarrow \langle \text{stmts} \rangle$
 $\langle \text{stmts} \rangle \rightarrow \langle \text{stmt} \rangle \mid \langle \text{stmt} \rangle ; \langle \text{stmts} \rangle$
 $\langle \text{stmt} \rangle \rightarrow \langle \text{var} \rangle = \langle \text{expr} \rangle$
 $\langle \text{var} \rangle \rightarrow a \mid b \mid c \mid d$
 $\langle \text{expr} \rangle \rightarrow \langle \text{term} \rangle + \langle \text{term} \rangle \mid \langle \text{term} \rangle - \langle \text{term} \rangle$
 $\langle \text{term} \rangle \rightarrow \langle \text{var} \rangle \mid \text{const}$

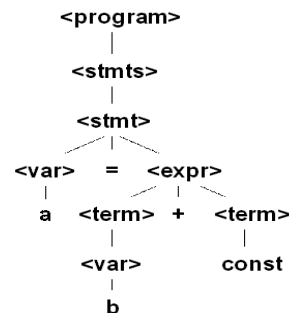
Parse Tree

A hierarchical representation of a derivation

An example derivation

$\langle \text{program} \rangle \Rightarrow \langle \text{stmts} \rangle$
 $\Rightarrow \langle \text{stmt} \rangle$
 $\Rightarrow \langle \text{var} \rangle = \langle \text{expr} \rangle$
 $\Rightarrow a = \langle \text{expr} \rangle$
 $\Rightarrow a = \langle \text{term} \rangle + \langle \text{term} \rangle$
 $\Rightarrow a = \langle \text{var} \rangle + \langle \text{term} \rangle$
 $\Rightarrow a = b + \langle \text{term} \rangle$
 $\Rightarrow a = b + \text{const}$

Figure 2.1 Parse Tree



Derivation

- Every string of symbols in the derivation is a sentential form
- A sentence is a sentential form that has only terminal symbols
- A leftmost derivation is one in which the leftmost nonterminal in each sentential form is the one that is expanded
- A derivation may be neither leftmost nor rightmost

Ambiguity in Grammars

- A grammar is *ambiguous* iff it generates a sentential form that has two or more distinct parse trees

An Unambiguous Expression Grammar

If we use the parse tree to indicate precedence levels of the operators, we cannot have ambiguity

$\langle \text{expr} \rangle \rightarrow \langle \text{expr} \rangle - \langle \text{term} \rangle \mid \langle \text{term} \rangle$
 $\langle \text{term} \rangle \rightarrow \langle \text{term} \rangle / \text{const} \mid \text{const}$

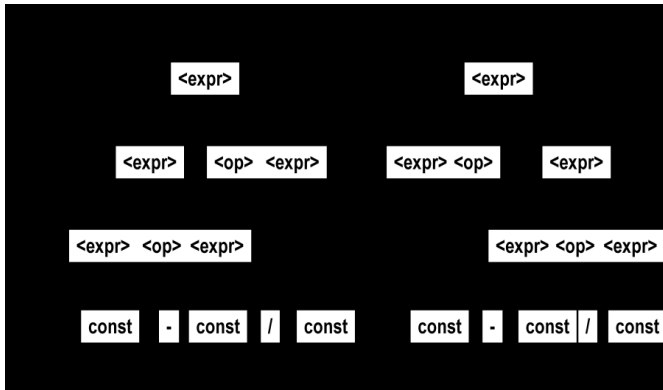


Figure 2.2 An Ambiguous Expression Grammar

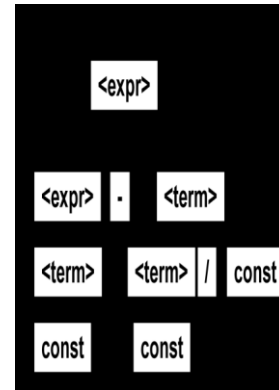


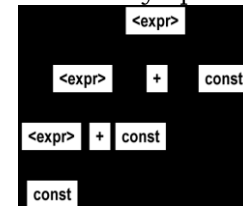
Figure 2.3 An Unambiguous Expression Grammar

Associativity of Operators

Operator associativity can also be indicated by a grammar

$\langle expr \rangle \rightarrow \langle expr \rangle + \langle expr \rangle \mid const$ (ambiguous)
 $\langle expr \rangle \rightarrow \langle expr \rangle + const \mid const$ (unambiguous)

Figure 2.4 Parse Tree for Associativity operator



1.9 Extended Backus-Naur Form (EBNF) – CO2

- Optional parts are placed in brackets ([])
 $\langle proc_call \rangle \rightarrow ident [(\langle expr_list \rangle)]$
- Alternative parts of RHSs are placed inside parentheses and separated via vertical bars
 $\langle term \rangle \rightarrow \langle term \rangle (+ \mid -) const$
- Repetitions (0 or more) are placed inside braces ({ })
 $\langle ident \rangle \rightarrow letter \{letter \mid digit\}$

BNF and EBNF

- BNF
 $\langle expr \rangle \rightarrow \langle expr \rangle + \langle term \rangle$
 $\quad \quad \quad \mid \langle expr \rangle - \langle term \rangle$
 $\quad \quad \quad \mid \langle term \rangle$
 $\langle term \rangle \rightarrow \langle term \rangle * \langle factor \rangle$
 $\quad \quad \quad \mid \langle term \rangle / \langle factor \rangle$
 $\quad \quad \quad \mid \langle factor \rangle$
- EBNF
 $\langle expr \rangle \rightarrow \langle term \rangle \{ (+ \mid -) \langle term \rangle \}$
 $\langle term \rangle \rightarrow \langle factor \rangle \{ (* \mid /) \langle factor \rangle \}$

1.10 Attribute Grammars – CO1, CO4

- Context-free grammars (CFGs) cannot describe all of the syntax of programming languages
- Additions to CFGs to carry some semantic info along parse trees
- Primary value of attribute grammars (AGs):
 - Static semantics specification
 - Compiler design (static semantics checking)

Definition

- An attribute grammar is a context-free grammar $G = (S, N, T, P)$ with the following additions:
 - For each grammar symbol x there is a set $A(x)$ of attribute values

- Each rule has a set of functions that define certain attributes of the nonterminals in the rule
- Each rule has a (possibly empty) set of predicates to check for attribute consistency
- Let $X_0 X_1 \dots X_n$ be a rule
- Functions of the form $S(X_0) = f(A(X_1), \dots, A(X_n))$ define *synthesized attributes*
- Functions of the form $I(X_j) = f(A(X_0), \dots, A(X_n))$, for $i \leq j \leq n$, define *inherited attributes*
- Initially, there are *intrinsic attributes* on the leaves

Example

- Syntax
 - $\langle \text{assign} \rangle \rightarrow \langle \text{var} \rangle = \langle \text{expr} \rangle$
 - $\langle \text{expr} \rangle \rightarrow \langle \text{var} \rangle + \langle \text{var} \rangle \mid \langle \text{var} \rangle$
 - $\langle \text{var} \rangle \rightarrow A \mid B \mid C$
- actual_type: synthesized for $\langle \text{var} \rangle$ and $\langle \text{expr} \rangle$
- expected_type: inherited for $\langle \text{expr} \rangle$
- Syntax rule $\langle \text{expr} \rangle \rightarrow \langle \text{var} \rangle[1] + \langle \text{var} \rangle[2]$
- Semantic rules $\langle \text{expr} \rangle.\text{actual_type} \rightarrow \langle \text{var} \rangle[1].\text{actual_type}$
- Predicate $\langle \text{var} \rangle[1].\text{actual_type} == \langle \text{var} \rangle[2].\text{actual_type}$
 $\langle \text{expr} \rangle.\text{expected_type} == \langle \text{expr} \rangle.\text{actual_type}$
- Syntax rule $\langle \text{var} \rangle \rightarrow \text{id}$
- Semantic rule $\langle \text{var} \rangle.\text{actual_type} \leftarrow \text{lookup}(\langle \text{var} \rangle.\text{string})$
- How are attribute values computed?
 - If all attributes were inherited, the tree could be decorated in top-down order.
 - If all attributes were synthesized, the tree could be decorated in bottom-up order.
 - In many cases, both kinds of attributes are used, and it is some combination of top-down and bottom-up that must be used.
 - $\langle \text{expr} \rangle.\text{expected_type} \leftarrow \text{inherited from parent}$
 - $\langle \text{var} \rangle[1].\text{actual_type} \leftarrow \text{lookup}(A)$
 - $\langle \text{var} \rangle[2].\text{actual_type} \leftarrow \text{lookup}(B)$
 - $\langle \text{var} \rangle[1].\text{actual_type} =? \langle \text{var} \rangle[2].\text{actual_type}$
 - $\langle \text{expr} \rangle.\text{actual_type} \leftarrow \langle \text{var} \rangle[1].\text{actual_type}$
 - $\langle \text{expr} \rangle.\text{actual_type} =? \langle \text{expr} \rangle.\text{expected_type}$

Describing the Meanings of Programs: Dynamic Semantics

- There is no single widely acceptable notation or formalism for describing semantics
- Operational Semantics
 - Describe the meaning of a program by executing its statements on a machine, either simulated or actual. The change in the state of the machine (memory, registers, etc.) defines the meaning of the statement
- To use operational semantics for a high-level language, a virtual machine is needed
- A hardware pure interpreter would be too expensive
- A software pure interpreter also has problems:
 - The detailed characteristics of the particular computer would make actions difficult to understand
 - Such a semantic definition would be machine- dependent

Operational Semantics

- A better alternative: A complete computer simulation
- The process:
 - Build a translator (translates source code to the machine code of an idealized computer)
 - Build a simulator for the idealized computer
- Evaluation of operational semantics:
 - Good if used informally (language manuals, etc.)
 - Extremely complex if used formally (**e.g.**, VDL), it was used for describing semantics of PL/I.
- Axiomatic Semantics
 - Based on formal logic (predicate calculus)
 - Original purpose: formal program verification
 - Approach: Define axioms or inference rules for each statement type in the language (to allow transformations of expressions to other expressions)
 - The expressions are called assertions

Axiomatic Semantics

- An assertion before a statement (a precondition) states the relationships and constraints among variables that are true at that point in execution
- An assertion following a statement is a postcondition
- A weakest precondition is the least restrictive precondition that will guarantee the postcondition
- Pre-post form: $\{P\}$ statement $\{Q\}$
- An example: $a = b + 1 \{a > 1\}$
- One possible precondition: $\{b > 10\}$
- Weakest precondition: $\{b > 0\}$
- Program proof process: The postcondition for the whole program is the desired result. Work back through the program to the first statement. If the precondition on the first statement is the same as the program spec, the program is correct.
- An axiom for assignment statements
 $(x = E):$
 $\{Qx \rightarrow E\} x = E \{Q\}$
- An inference rule for sequences
 - For a sequence $S1; S2:$
 - $\{P1\} S1 \{P2\}$
 - $\{P2\} S2 \{P3\}$
- An inference rule for logical pretest loops
For the loop construct:
 $\{P\}$ while B do S end $\{Q\}$
Characteristics of the loop invariant
I must meet the following conditions:
 - $P \Rightarrow I$ (the loop invariant must be true initially)
 - $\{I\} B \{I\}$ (evaluation of the Boolean must not change the validity of I)
 - $\{I \text{ and } B\} S \{I\}$ (I is not changed by executing the body of the loop)
 - $(I \text{ and } (\text{not } B)) \Rightarrow Q$ (if I is true and B is false, Q is implied)
 - The loop terminates (this can be difficult to prove)
- The loop invariant I is a weakened version of the loop postcondition, and it is also a precondition.

- I must be weak enough to be satisfied prior to the beginning of the loop, but when combined with the loop exit condition, it must be strong enough to force the truth of the postcondition.

Evaluation of Axiomatic Semantics:

- Developing axioms or inference rules for all of the statements in a language is difficult
- It is a good tool for correctness proofs, and an excellent framework for reasoning about programs, but it is not as useful for language users and compiler writers
- Its usefulness in describing the meaning of a programming language is limited for language users or compiler writers

Denotational Semantics

- Based on recursive function theory
- The most abstract semantics description method
- Originally developed by Scott and Strachey (1970)
- The process of building a denotational spec for a language (not necessarily easy):
- Define a mathematical object for each language entity
- Define a function that maps instances of the language entities onto instances of the corresponding mathematical objects
- The meaning of language constructs are defined by only the values of the program's variables
- The difference between denotational and operational semantics: In operational semantics, the state changes are defined by coded algorithms; in denotational semantics, they are defined by rigorous mathematical functions
- The state of a program is the values of all its current variables

$$s = \{ \langle i1, v1 \rangle, \langle i2, v2 \rangle, \dots, \langle in, vn \rangle \}$$
- Let VARMAP be a function that, when given a variable name and a state, returns the current value of the variable

$$\text{VARMAP}(ij, s) = vj$$
- Decimal Numbers
 - The following denotational semantics description maps decimal numbers as strings of symbols into numeric values

$$\langle \text{dec_num} \rangle \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

$$\Rightarrow \langle \text{dec_num} \rangle (0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9)$$

$$\text{Mdec}('0') = 0, \text{Mdec}('1') = 1, \dots, \text{Mdec}('9') = 9$$

$$\text{Mdec}(\langle \text{dec_num} \rangle '0') = 10 * \text{Mdec}(\langle \text{dec_num} \rangle)$$

$$\text{Mdec}(\langle \text{dec_num} \rangle '1') = 10 * \text{Mdec}(\langle \text{dec_num} \rangle) + 1$$

$$\dots$$

$$\text{Mdec}(\langle \text{dec_num} \rangle '9') = 10 * \text{Mdec}(\langle \text{dec_num} \rangle) + 9$$

Expressions

- Map expressions onto $Z \cup \{\text{error}\}$
- We assume expressions are decimal numbers, variables, or binary expressions having one arithmetic operator and two operands, each of which can be an expression
- Assignment Statements
 - Maps state sets to state sets
- Logical Pretest Loops
 - Maps state sets to state sets

- The meaning of the loop is the value of the program variables after the statements in the loop have been executed the prescribed number of times, assuming there have been no errors
- In essence, the loop has been converted from iteration to recursion, where the recursive control is mathematically defined by other recursive state mapping functions
- Recursion, when compared to iteration, is easier to describe with mathematical rigor
- Evaluation of denotational semantics
 - Can be used to prove the correctness of programs
 - Provides a rigorous way to think about programs
 - Can be an aid to language design
 - Has been used in compiler generation systems
 - Because of its complexity, they are of little use to language users

Summary

- BNF and context-free grammars are equivalent meta-languages
 - Well-suited for describing the syntax of programming languages
- An attribute grammar is a descriptive formalism that can describe both the syntax and the semantics of a language
- Three primary methods of semantics description
 - Operation, axiomatic, denotational

UNIT-2

Data Types and Variables

Introduction

- A *data type* defines a collection of data objects and a set of predefined operations on those objects
- A *descriptor* is the collection of the attributes of a variable
- An *object* represents an instance of a user-defined (abstract data) type
- One design issue for all data types: What operations are defined and how are they specified?

2.1 Primitive Data Types – CO2

- Almost all programming languages provide a set of *primitive data types*
- Primitive data types: Those not defined in terms of other data types
- Some primitive data types are merely reflections of the hardware
- Others require only a little non-hardware support for their implementation

Integer

- Almost always an exact reflection of the hardware so the mapping is trivial
- There may be as many as eight different integer types in a language
- Java's signed integer sizes: byte, short, int, long

Floating Point

- Model real numbers, but only as approximations
- Languages for scientific use support at least two floating-point types (**e.g.**, float and double; sometimes more)
- Usually exactly like the hardware, but not always
- IEEE Floating-Point Standard 754

Complex

- Some languages support a complex type, **e.g.**, Fortran and Python
- Each value consists of two floats, the real part and the imaginary part
- Literal form (in Python):
(7 + 3j), where 7 is the real part and 3 is the imaginary part

Decimal

- For business applications (money)
 - Essential to COBOL
 - C# offers a decimal data type
- Store a fixed number of decimal digits, in coded form (BCD)
- *Advantage*: accuracy
- *Disadvantages*: limited range, wastes memory

Boolean

- Simplest of all
- Range of values: two elements, one for “true” and one for “false”
- Could be implemented as bits, but often as bytes
 - Advantage: readability

Character

- Stored as numeric coding
- Most commonly used coding: ASCII
- An alternative, 16-bit coding: Unicode

- Includes characters from most natural languages
- Originally used in Java
- C# and JavaScript also support Unicode

Character String Types

- Values are sequences of characters
- Design issues:
 - Is it a primitive type or just a special kind of array?
 - Should the length of strings be static or dynamic?

Operations

- Typical operations:
 - Assignment and copying
 - Comparison (=, >, etc.)
 - Catenation
 - Substring reference
 - Pattern matching

Character String Type in Certain Languages

- C and C++
 - Not primitive
 - Use **char** arrays and a library of functions that provide operations
- SNOBOL4 (a string manipulation language)
 - Primitive
 - Many operations, including elaborate pattern matching
- Fortran and Python
 - Primitive type with assignment and several operations
- Java
 - Primitive via the String class
- Perl, JavaScript, Ruby, and PHP
 - Provide built-in pattern matching, using regular expressions

Character String Length Options

- Static: COBOL, Java's String class
- *Limited Dynamic Length*: C and C++
 - In these languages, a special character is used to indicate the end of a string's characters, rather than maintaining the length
- *Dynamic* (no maximum): SNOBOL4, Perl, JavaScript
- Ada supports all three string length options

Evaluation

- Aid to writability
- As a primitive type with static length, they are inexpensive to provide—why not have them?
- Dynamic length is nice, but is it worth the expense?

Implementation

- Static length: compile-time descriptor
- Limited dynamic length: may need a run-time descriptor for length (but not in C and C++)
- Dynamic length: need run-time descriptor; allocation/de-allocation is the biggest implementation problem

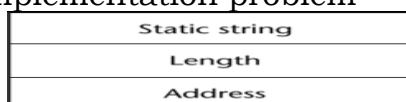


Figure 3.1 Compile-Time Descriptor

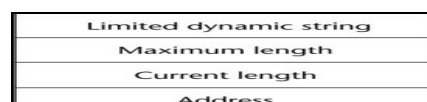


Figure 3.2 Run-Time Descriptors

2.2 User-Defined Ordinal Types - CO2

- An ordinal type is one in which the range of possible values can be easily associated with the set of positive integers
- Examples of primitive ordinal types in Java
 - integer
 - char
 - boolean

Enumeration Types

- All possible values, which are named constants, are provided in the definition
- C# example

```
enum days {mon, tue, wed, thu, fri, sat, sun};
```
- Design issues
 - Is an enumeration constant allowed to appear in more than one type definition, and if so, how is the type of an occurrence of that constant checked?
 - Are enumeration values coerced to integer?
 - Any other type coerced to an enumeration type?

Evaluation of Enumerated Type

- Aid to readability, **e.g.**, no need to code a color as a number
- Aid to reliability, **e.g.**, compiler can check:
 - Operations (don't allow colors to be added)
 - No enumeration variable can be assigned a value outside its defined range
 - Ada, C#, and Java 5.0 provide better support for enumeration than C++ because enumeration type variables in these languages are not coerced into integer types

Subrange Types

- An ordered contiguous subsequence of an ordinal type
 - Example: 12..18 is a subrange of integer type
- Ada's design

```
type Days is (mon, tue, wed, thu, fri, sat, sun);
subtype Weekdays is Days range mon..fri;
subtype Index is Integer range 1..100;
```

```
Day1: Days;
Day2: Weekday;
Day2 := Day1;
```

Subrange Evaluation

- Aid to readability
 - Make it clear to the readers that variables of subrange can store only certain range of values
- Reliability
 - Assigning a value to a subrange variable that is outside the specified range is detected as an error

Implementation

- Enumeration types are implemented as integers
- Subrange types are implemented like the parent types with code inserted (by the compiler) to restrict assignments to subrange variables

Array Types

- An array is an aggregate of homogeneous data elements in which an individual element is identified by its position in the aggregate, relative to the first element.

Array Design Issues

- What types are legal for subscripts?
- Are subscripting expressions in element references range checked?
- When are subscript ranges bound?
- When does allocation take place?
- What is the maximum number of subscripts?
- Can array objects be initialized?
- Are any kind of slices supported?

Array Indexing

- *Indexing* (or subscripting) is a mapping from indices to elements
array_name (index_value_list) → an element
- Index Syntax
 - FORTRAN, PL/I, Ada use parentheses
- Ada explicitly uses parentheses to show uniformity between array references and function calls because both are *mappings*
 - Most other languages use brackets

Arrays Index (Subscript) Types

- FORTRAN, C: integer only
- Ada: integer or enumeration (includes Boolean and char)
- Java: integer types only
- Index range checking
 - C, C++, Perl, and Fortran do not specify range checking
 - Java, ML, C# specify range checking
 - In Ada, the default is to require range checking, but it can be turned off

Subscript Binding and Array Categories

- *Static*: subscript ranges are statically bound and storage allocation is static (before run-time)
 - Advantage: efficiency (no dynamic allocation)
- *Fixed stack-dynamic*: subscript ranges are statically bound, but the allocation is done at declaration time
 - Advantage: space efficiency
- *Stack-dynamic*: subscript ranges are dynamically bound and the storage allocation is dynamic (done at run-time)
 - Advantage: flexibility (the size of an array need not be known until the array is to be used)
- *Fixed heap-dynamic*: similar to fixed stack-dynamic: storage binding is dynamic but fixed after allocation (i.e., binding is done when requested and storage is allocated from heap, not stack)
- *Heap-dynamic*: binding of subscript ranges and storage allocation is dynamic and can change any number of times
 - Advantage: flexibility (arrays can grow or shrink during program execution)
- C and C++ arrays that include static modifier are static
- C and C++ arrays without static modifier are fixed stack-dynamic
- C and C++ provide fixed heap-dynamic arrays
- C# includes a second array class `ArrayList` that provides fixed heap-dynamic
- Perl, JavaScript, Python, and Ruby support heap-dynamic arrays

Array Initialization

- Some language allow initialization at the time of storage allocation
 - C, C++, Java, C# example
`int list [] = {4, 5, 7, 83}`
 - Character strings in C and C++
`char name [] = "freddie";`
 - Arrays of strings in C and C++
`char *names [] = {"Bob", "Jake", "Joe"};`
 - Java initialization of String objects
`String[] names = {"Bob", "Jake", "Joe"};`

Heterogeneous Arrays

- A *heterogeneous array* is one in which the elements need not be of the same type
- Supported by Perl, Python, JavaScript, and Ruby

Arrays Operations

- APL provides the most powerful array processing operations for vectors and matrixes as well as unary operators (for example, to reverse column elements)
- Ada allows array assignment but also catenation
- Python's array assignments, but they are only reference changes. Python also supports array catenation and element membership operations
- Ruby also provides array catenation
- Fortran provides *elemental* operations because they are between pairs of array elements
 - For example, + operator between two arrays results in an array of the sums of the element pairs of the two arrays

Rectangular and Jagged Arrays

- A rectangular array is a multi-dimensioned array in which all of the rows have the same number of elements and all columns have the same number of elements
- A jagged matrix has rows with varying number of elements
 - Possible when multi-dimensioned arrays actually appear as arrays of arrays
- C, C++, and Java support jagged arrays
- Fortran, Ada, and C# support rectangular arrays (C# also supports jagged arrays)

Slices

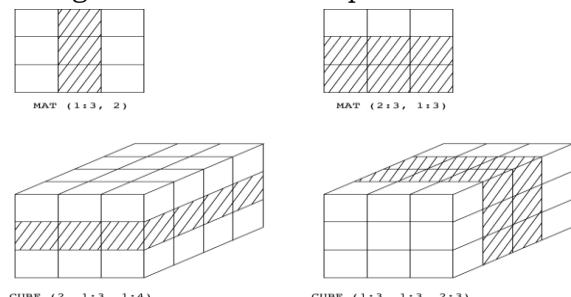
- A slice is some substructure of an array; nothing more than a referencing mechanism
- Slices are only useful in languages that have array operations

Slice Examples

- Fortran 95
 - Integer, Dimension (10) :: Vector
 - Integer, Dimension (3, 3) :: Mat
 - Integer, Dimension (3, 3) :: Cube

Vector (3:6) is a four element array

Figure 3.3 Slices Examples in Fortran 95



Implementation of Arrays

- Access function maps subscript expressions to an address in the array
- Access function for single-dimensioned arrays:
`address(list[k]) = address (list[lower_bound]) + ((k-lower_bound) * element_size)`

Accessing Multi-dimensional Arrays

- Two common ways:
 - Row major order (by rows) – used in most languages
 - Column major order (by columns) – used in Fortran

Figure 3.4 Locating an Element in a Multi-dimensional Array

	1	2	...	j-1	j	...	n
1							
2							
⋮							
i-1							
i					⊗		
⋮							
m							

Compile-Time Descriptors

Array
Element type
Index type
Index lower bound
Index upper bound
Address

Figure 3.5 Single-Dimensioned Array

Multidimensioned array
Element type
Index type
Number of dimensions
Index range 1
⋮
Index range <i>n</i>
Address

Figure 3.6 Multi-Dimensioned Array

Associative Arrays

- An *associative array* is an unordered collection of data elements that are indexed by an equal number of values called *keys*
 - User-defined keys must be stored
- Design issues:
 - What is the form of references to elements?
 - Is the size static or dynamic?

Associative Arrays in Perl

- Names begin with **%**; literals are delimited by parentheses
`%hi_temps = ("Mon" => 77, "Tue" => 79, "Wed" => 65, ...);`
- Subscripting is done using braces and keys
`$hi_temps{"Wed"} = 83;`
- Elements can be removed with delete
`delete $hi_temps{"Tue"};`

2.3 Record Types – CO2

- A *record* is a possibly heterogeneous aggregate of data elements in which the individual elements are identified by names
- Design issues:
 - What is the syntactic form of references to the field?
 - Are elliptical references allowed?

Definition of Records in COBOL

- COBOL uses level numbers to show nested records; others use recursive definition

```
01 EMP-REC.
  02 EMP-NAME.
    05 FIRST PIC X(20).
    05 MID PIC X(10).
    05 LAST PIC X(20).
  02 HOURLY-RATE PIC 99V99.
```

Definition of Records in Ada

- Record structures are indicated in an orthogonal way

```
type Emp_Rec_Type is record
  First: String (1..20);
  Mid: String (1..10);
  Last: String (1..20);
  Hourly_Rate: Float;
end record;
Emp_Rec: Emp_Rec_Type;
```

References to Records

- Record field references
 - COBOL
field_name OF record_name_1 OF ... OF record_name_n
 - Others (dot notation)
record_name_1.record_name_2. ... record_name_n.field_name
- Fully qualified references must include all record names
- Elliptical references allow leaving out record names as long as the reference is unambiguous, for example in COBOL
FIRST, FIRST OF EMP-NAME, and FIRST of EMP-REC are elliptical references to the employee's first name

Operations on Records

- Assignment is very common if the types are identical
- Ada allows record comparison
- Ada records can be initialized with aggregate literals
- COBOL provides MOVE CORRESPONDING
 - Copies a field of the source record to the corresponding field in the target record

Evaluation and Comparison to Arrays

- Records are used when collection of data values is heterogeneous
- Access to array elements is much slower than access to record fields, because subscripts are dynamic (field names are static)
- Dynamic subscripts could be used with record field access, but it would disallow type checking and it would be much slower

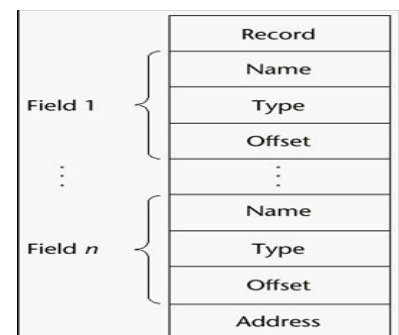


Figure 3.7 Implementation of Record Type

Unions Types

- A *union* is a type whose variables are allowed to store different type values at different times during execution
- Design issues
 - Should type checking be required?
 - Should unions be embedded in records?

Discriminated vs. Free Unions

- Fortran, C, and C++ provide union constructs in which there is no language support for type checking; the union in these languages is called *free union*
- Type checking of unions require that each union include a type indicator called a *discriminant*
 - Supported by Ada

Ada Union Types

```

type Shape is (Circle, Triangle, Rectangle);
type Colors is (Red, Green, Blue);
type Figure (Form: Shape) is record
    Filled: Boolean;
    Color: Colors;
    case Form is
        when Circle => Diameter: Float;
        when Triangle => Leftside, Rightside: Integer;
        Angle: Float;
        when Rectangle => Side1, Side2: Integer;
    end case;
end record;

```

Ada Union Type Illustrated

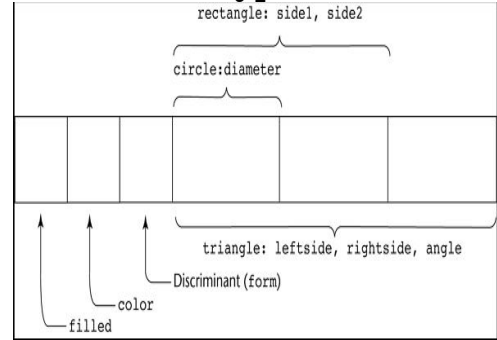


Figure 3.8 A Discriminated Union of Three Shape Variables

Evaluation of Unions

- Free unions are unsafe
 - Do not allow type checking
- Java and C# do not support unions
 - Reflective of growing concerns for safety in programming language
- Ada's discriminated unions are safe

Pointer and Reference Types

- A *pointer* type variable has a range of values that consists of memory addresses and a special value, *nil*
- Provide the power of indirect addressing
- Provide a way to manage dynamic memory
- A pointer can be used to access a location in the area where storage is dynamically created (usually called a *heap*)

Design Issues of Pointers

- What are the scope of and lifetime of a pointer variable?
- What is the lifetime of a heap-dynamic variable?
- Are pointers restricted as to the type of value to which they can point?
- Are pointers used for dynamic storage management, indirect addressing, or both?
- Should the language support pointer types, reference types, or both?

Pointer Operations

- Two fundamental operations: assignment and dereferencing
- Assignment is used to set a pointer variable's value to some useful address
- Dereferencing yields the value stored at the location represented by the pointer's value
 - Dereferencing can be explicit or implicit
 - C++ uses an explicit operation via `*`

$j = *ptr$

sets j to the value located at ptr

Pointer Assignment Illustration

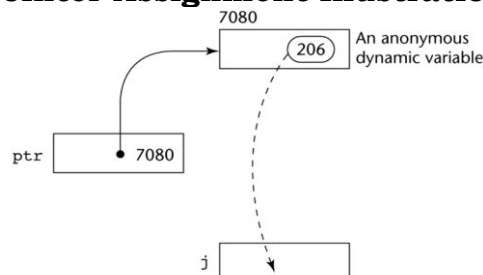


Figure 3.9 The assignment operation $j = *ptr$

Problems with Pointers

- Dangling pointers (dangerous)
 - A pointer points to a heap-dynamic variable that has been deallocated
- Lost heap-dynamic variable
 - An allocated heap-dynamic variable that is no longer accessible to the user program (often called *garbage*)
- Pointer p1 is set to point to a newly created heap-dynamic variable
- Pointer p1 is later set to point to another newly created heap-dynamic variable
- The process of losing heap-dynamic variables is called *memory leakage*

Pointers in Ada

- Some dangling pointers are disallowed because dynamic objects can be automatically deallocated at the end of pointer's type scope
- The lost heap-dynamic variable problem is not eliminated by Ada (possible with UNCHECKED_DEALLOCATION)

Pointers in C and C++

- Extremely flexible but must be used with care
- Pointers can point at any variable regardless of when or where it was allocated
- Used for dynamic storage management and addressing
- Pointer arithmetic is possible
- Explicit dereferencing and address-of operators
- Domain type need not be fixed (**void ***)
 - void * can point to any type and can be type checked (cannot be de-referenced)

Pointer Arithmetic in C and C++

```
float stuff[100];  
float *p;  
p = stuff;
```

*(p+5) is equivalent to stuff[5] and p[5]

*(p+i) is equivalent to stuff[i] and p[i]

Reference Types

- C++ includes a special kind of pointer type called a *reference type* that is used primarily for formal parameters
 - Advantages of both pass-by-reference and pass-by-value
- Java extends C++'s reference variables and allows them to replace pointers entirely
 - References are references to objects, rather than being addresses
- C# includes both the references of Java and the pointers of C++

Evaluation of Pointers

- Dangling pointers and dangling objects are problems as is heap management
- Pointers are like goto's--they widen the range of cells that can be accessed by a variable
- Pointers or references are necessary for dynamic data structures--so we can't design a language without them

Representations of Pointers

- Large computers use single values
- Intel microprocessors use segment and offset

Dangling Pointer Problem

- *Tombstone*: extra heap cell that is a pointer to the heap-dynamic variable
 - The actual pointer variable points only at tombstones
 - When heap-dynamic variable de-allocated, tombstone remains but set to nil
 - Costly in time and space

- *Locks-and-keys*: Pointer values are represented as (key, address) pairs
 - Heap-dynamic variables are represented as variable plus cell for integer lock value
 - When heap-dynamic variable allocated, lock value is created and placed in lock cell and key cell of pointer

Heap Management

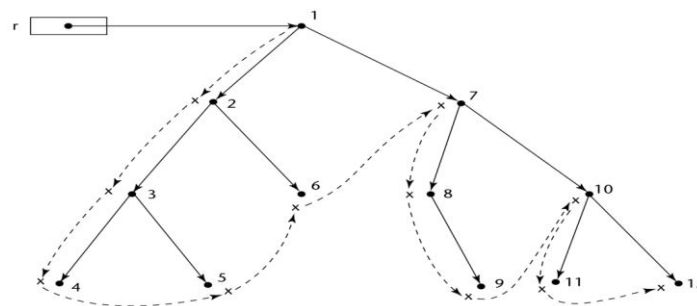
- A very complex run-time process
- Single-size cells vs. variable-size cells
- Two approaches to reclaim garbage
 - Reference counters (*eager approach*): reclamation is gradual
 - Mark-sweep (*lazy approach*): reclamation occurs when the list of variable space becomes empty

Reference Counter

- Reference counters: maintain a counter in every cell that store the number of pointers currently pointing at the cell
 - *Disadvantages*: space required, execution time required, complications for cells connected circularly
 - *Advantage*: it is intrinsically incremental, so significant delays in the application execution are avoided

Mark-Sweep

- The run-time system allocates storage cells as requested and disconnects pointers from cells as necessary; mark-sweep then begins
 - Every heap cell has an extra bit used by collection algorithm
 - All cells initially set to garbage
 - All pointers traced into heap, and reachable cells marked as not garbage
 - All garbage cells returned to list of available cells
 - *Disadvantages*: in its original form, it was done too infrequently. When done, it caused significant delays in application execution. Contemporary markswEEP algorithms avoid this by doing it more often—called incremental markswEEP



Dashed lines show the order of node marking
Figure 3.10 Marking Algorithm

Variable-Size Cells

- All the difficulties of single-size cells plus more
- Required by most programming languages
- If mark-sweep is used, additional problems occur
 - The initial setting of the indicators of all cells in the heap is difficult
 - The marking process is nontrivial
 - Maintaining the list of available space is another source of overhead